

Instituto Politécnico Nacional
Escuela Superior de Comercio y Administración
Unidad Santo Tomás

Definición de la Pila Tecnológica

Integrantes:

- Escobar Hernández Edna
- González Torres Fernanda
- López Reyes Fernanda
- Moreno Ruiz Diana Yael

Profesor: Jovan del Prado.

Ciclo escolar 2026/1

Miércoles 15 de abril de 2026

Definición del Framework y Tecnologías del Proyecto Pugnela

El proyecto Pugnela implementa un sistema de carrito de compras con arquitectura modular, dividido en dos componentes principales: módulo administrativo para gestión completa de productos, usuarios y pedidos; y módulo de usuario final para navegación, selección de productos y proceso de compra. Esta estructura asegura eficiencia operativa, escalabilidad y mantenimiento simplificado mediante tecnologías probadas y estandarizadas.

Tecnologías Seleccionadas

Servidor Web: Apache HTTP Server:

Se selecciona Apache por su reconocida estabilidad, robustez en seguridad y capacidad para procesar múltiples solicitudes concurrentes, característica esencial en entornos de comercio electrónico con alta interacción simultánea de usuarios.

Lenguaje de Programación: PHP

PHP se emplea como lenguaje principal por su madurez, versatilidad y perfecta integración con tecnologías web estándar. Permite desarrollar de forma unificada ambos módulos del sistema, desde la gestión administrativa hasta la experiencia de usuario final.

Base de Datos: MySQL

MySQL ofrece rendimiento óptimo para operaciones transaccionales frecuentes, ideal para gestionar catálogos de productos, registros de usuarios, historial de pedidos y datos de transacciones con integridad referencial garantizada.

Herramientas de Soporte al Desarrollo

Inteligencia Artificial: Claude

Claude actúa como asistente principal en codificación, depuración, diseño de base de datos y optimización de algoritmos, acelerando significativamente las iteraciones de desarrollo mediante generación automática de código y resolución de incidencias.

Control de Versiones: GitHub

GitHub centraliza el repositorio del código fuente, documentación y artefactos del proyecto, facilitando colaboración en equipo, trazabilidad de cambios y despliegues continuos con integridad garantizada.

Gestión de Proyecto: Microsoft Project

Se utiliza Microsoft Project para elaborar diagramas de Gantt detallados, asignar recursos, establecer hitos temporales y monitorear el avance global del desarrollo del sistema Pugnela.

Estructura de Carpetas Principal

- **admin/** → Módulo administrativo del sistema
 - controllers/ → Controladores del panel administrativo
 - models/ → Modelos de base de datos para administración
 - views/ → Interfaces visuales del administrador
 - assets/
 - css/ → Estilos del panel admin
 - js/ → Scripts del panel admin
 - img/ → Imágenes del módulo admin
 - includes/
 - header.php
 - sidebar.php
 - footer.php
 - index.php → Página principal del administrador
- **user/** → Módulo de usuario final / tienda virtual
 - controllers/ → Controladores del cliente
 - models/ → Modelos de datos del usuario
 - views/ → Interfaces del usuario final
 - assets/

- ☐
 - `css/` → Estilos tienda virtual
 - `js/` → Scripts tienda virtual
 - `img/` → Imágenes públicas
- `includes/`
 - `header.php`
 - `navbar.php`
 - `footer.php`
- `index.php` → Página principal de la tienda
- ☐ **`config/`** → Configuración general del sistema
 - `database.php` → Conexión con MySQL
 - `constants.php` → Variables globales y rutas
- ☐ **`core/`** → Funciones principales del sistema
 - `Router.php`
 - `Auth.php`
 - `Session.php`
 - `Functions.php`
- ☐ **`uploads/`** → Imágenes de productos y archivos cargados
- ☐ **`logs/`** → Registro de errores y eventos del sistema
- ☐ **`docs/`** → Documentación técnica del proyecto
- ☐ **`tests/`** → Pruebas del sistema
- ☐ **`index.php`** → Archivo principal de entrada
- ☐ **`.htaccess`** → Configuración y seguridad en Apache
- ☐ **`README.md`** → Información general del proyecto en GitHub

Identificar las entidades, relaciones y atributos.

1. Usuario / Cliente

- idUsuario (PK, entero autonumérico)
- nombre (texto, no null)
- apellido (texto, opcional o no null según necesidad)
- email (texto único, no null)
- password (texto, almacenado hasheado)
- telefono (texto)
- direccion (texto, dirección completa o parcial)
- cp (código postal, texto o entero)
- ciudad (texto)
- estado (texto)
- fechaRegistro (datetime)

2. Producto

- idProducto (PK)
- nombre (texto, no null)
- descripcion (texto largo, opcional)
- precio (decimal, no null)
- tipo (texto: 'clásico' o 'personalizado')
- imagenUrl (texto, ruta o URL de la imagen)
- activo (booleano, marca si se muestra en el catálogo)

3. Carrito de compras

- idCarrito (PK)
- idUsuario (FK a Usuario)
- fechaCreacion (datetime)
- fechaActualizacion (datetime, cada vez que se modifica)

4. Carrito_Producto (línea de carrito)

- idCarritoProducto (PK)
- idCarrito (FK a Carrito)
- idProducto (FK a Producto)

- cantidad (entero, no null)
- precioUnitario (decimal, valor en ese momento, útil si el precio cambia)

5. Pedido / Orden

- idPedido (PK)
- idUsuario (FK a Usuario)
- fechaPedido (datetime)
- total (decimal, suma de todos los productos + impuestos si aplica)
- estado (texto: 'pendiente', 'pagado', 'enviado', etc.)

6. Pedido_Producto (detalle de pedido)

- idPedidoProducto (PK)
- idPedido (FK a Pedido)
- idProducto (FK a Producto)
- cantidad (entero, no null)
- precioUnitarioEnCompra (decimal, precio grabado en la compra)

7. Pago

- idPago (PK)
- idPedido (FK a Pedido)
- metodoPago (texto: 'tarjeta', 'oxxo', 'transferencia', etc.)
- monto (decimal, igual al total del pedido normalmente)
- fechaPago (datetime)
- referencia (texto, número de transacción o folio)
- estatusPago (texto: 'pendiente', 'aprobado', 'rechazado')

Relación	Cómo se conecta
<i>Usuario – Carrito</i>	Un usuario tiene un carrito activo (1:1 o 1:N según diseño).
<i>Carrito – Carrito_Producto</i>	Un carrito tiene N renglones de productos.
<i>Producto – Carrito_Producto</i>	Un producto puede estar en varios carritos.
<i>Usuario – Pedido</i>	Un usuario tiene varios pedidos.
<i>Pedido – Pedido_Producto</i>	Un pedido tiene varios productos.
<i>Producto – Pedido_Producto</i>	Un producto puede aparecer en varios pedidos.
<i>Pedido – Pago</i>	Un pedido tiene un pago asociado (o varios, si permites pagos parciales).

1. Diagrama Entidad–Relación

erDiagram

USUARIO ||--|| CARRITO : tiene

USUARIO ||--o{ PEDIDO : realiza

CARRITO ||--o{ CARRITO_PRODUCTO : contiene

PRODUCTO ||--o{ CARRITO_PRODUCTO : aparece_en

PEDIDO ||--o{ PEDIDO_PRODUCTO : incluye

PRODUCTO ||--o{ PEDIDO_PRODUCTO : se_vende_en

PEDIDO ||--|| PAGO : genera

USUARIO {

int idUsuario PK

varchar nombre

varchar apellido

varchar email

varchar password

varchar telefono

varchar direccion

```
    varchar cp
    varchar ciudad
    varchar estado
    datetime fechaRegistro
}
```

```
PRODUCTO {
    int idProducto PK
    varchar nombre
    text descripcion
    decimal precio
    varchar tipo
    varchar imagenUrl
    boolean activo
}
```

```
CARRITO {
    int idCarrito PK
    int idUsuario FK
    datetime fechaCreacion
    datetime fechaActualizacion
}
```

```
CARRITO_PRODUCTO {
    int idCarritoProducto PK
    int idCarrito FK
    int idProducto FK
    int cantidad
}
```



```
    decimal precioUnitario  
}
```

```
PEDIDO {  
    int idPedido PK  
    int idUsuario FK  
    datetime fechaPedido  
    decimal total  
    varchar estado  
}
```

```
PEDIDO_PRODUCTO {  
    int idPedidoProducto PK  
    int idPedido FK  
    int idProducto FK  
    int cantidad  
    decimal precioUnitarioEnCompra  
}
```

```
PAGO {  
    int idPago PK  
    int idPedido FK  
    varchar metodoPago  
    decimal monto  
    datetime fechaPago  
    varchar referencia  
    varchar estatusPago  
}
```

2. Relación con entidades

Relación	Tipo	Explicación
Usuario – Carrito	1:1	Cada usuario tiene un carrito activo.
Usuario – Pedido	1:N	Un usuario puede realizar varios pedidos.
Carrito – Carrito_Producto	1:N	Un carrito puede contener varios productos.
Producto Carrito_Producto	– 1:N	Un producto puede estar en distintos carritos.
Pedido – Pedido_Producto	1:N	Un pedido puede contener varios productos.
Producto Pedido_Producto	– 1:N	Un producto puede aparecer en distintos pedidos.
Pedido – Pago	1:1	Cada pedido genera un pago asociado.

3. Modelo relacional en 4FN

USUARIO

Usuario(idUsuario, nombre, apellido, email, password, telefono, direccion, cp, ciudad, estado, fechaRegistro)

PRODUCTO

Producto(idProducto, nombre, descripcion, precio, tipo, imagenUrl, activo)

CARRITO

Carrito(idCarrito, idUsuario, fechaCreacion, fechaActualizacion)

CARRITO_PRODUCTO

Carrito_Producto(idCarritoProducto, idCarrito, idProducto, cantidad, precioUnitario)

PEDIDO

Pedido(idPedido, idUsuario, fechaPedido, total, estado)

PEDIDO_PRODUCTO

Pedido_Producto(idPedidoProducto, idPedido, idProducto, cantidad, precioUnitarioEnCompra)

PAGO

Pago(idPago, idPedido, metodoPago, monto, fechaPago, referencia, estatusPago)

4. Modelo físico / SQL MySQL

```
CREATE TABLE Usuario (  
    idUsuario INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    apellido VARCHAR(100),  
    email VARCHAR(150) NOT NULL UNIQUE,  
    password VARCHAR(255) NOT NULL,  
    telefono VARCHAR(20),  
    direccion VARCHAR(255),  
    cp VARCHAR(10),  
    ciudad VARCHAR(100),  
    estado VARCHAR(100),  
    fechaRegistro DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE Producto (  
    idProducto INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(150) NOT NULL,  
    descripcion TEXT,  
    precio DECIMAL(10,2) NOT NULL,  
    tipo VARCHAR(50) NOT NULL,  
    imagenUrl VARCHAR(255),  
    activo BOOLEAN DEFAULT TRUE  
);
```

```
CREATE TABLE Carrito (  
    idCarrito INT AUTO_INCREMENT PRIMARY KEY,  
    idUsuario INT NOT NULL,
```

```
    fechaCreacion DATETIME DEFAULT CURRENT_TIMESTAMP,  
    fechaActualizacion DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (idUserio) REFERENCES Usuario(idUsuario)  
);
```

```
CREATE TABLE Carrito_Producto (  
    idCarritoProducto INT AUTO_INCREMENT PRIMARY KEY,  
    idCarrito INT NOT NULL,  
    idProducto INT NOT NULL,  
    cantidad INT NOT NULL,  
    precioUnitario DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (idCarrito) REFERENCES Carrito(idCarrito),  
    FOREIGN KEY (idProducto) REFERENCES Producto(idProducto)  
);
```

```
CREATE TABLE Pedido (  
    idPedido INT AUTO_INCREMENT PRIMARY KEY,  
    idUsuario INT NOT NULL,  
    fechaPedido DATETIME DEFAULT CURRENT_TIMESTAMP,  
    total DECIMAL(10,2) NOT NULL,  
    estado VARCHAR(50) DEFAULT 'pendiente',  
    FOREIGN KEY (idUserio) REFERENCES Usuario(idUsuario)  
);
```

```
CREATE TABLE Pedido_Producto (  
    idPedidoProducto INT AUTO_INCREMENT PRIMARY KEY,  
    idPedido INT NOT NULL,  
    idProducto INT NOT NULL,
```

```

cantidad INT NOT NULL,
precioUnitarioEnCompra DECIMAL(10,2) NOT NULL,
FOREIGN KEY (idPedido) REFERENCES Pedido(idPedido),
FOREIGN KEY (idProducto) REFERENCES Producto(idProducto)
);

```

```

CREATE TABLE Pago (
    idPago INT AUTO_INCREMENT PRIMARY KEY,
    idPedido INT NOT NULL,
    metodoPago VARCHAR(50) NOT NULL,
    monto DECIMAL(10,2) NOT NULL,
    fechaPago DATETIME DEFAULT CURRENT_TIMESTAMP,
    referencia VARCHAR(100),
    estatusPago VARCHAR(50) DEFAULT 'pendiente',
    FOREIGN KEY (idPedido) REFERENCES Pedido(idPedido)
);

```

5. Diccionario de datos de campos INT

Tabla	Campo INT	Uso
<i>Usuario</i>	idUsuario	Identifica de forma única a cada cliente.
<i>Producto</i>	idProducto	Identifica cada producto del catálogo.
<i>Carrito</i>	idCarrito	Identifica el carrito de compras.
<i>Carrito</i>	idUsuario	Relaciona el carrito con el usuario.
<i>Carrito_Producto</i>	idCarritoProducto	Identifica cada línea del carrito.
<i>Carrito_Producto</i>	idCarrito	Relaciona el producto con un carrito.
<i>Carrito_Producto</i>	idProducto	Relaciona el carrito con un producto.
<i>Carrito_Producto</i>	cantidad	Indica cuántas piezas quiere comprar el usuario.

<i>Pedido</i>	idPedido	Identifica cada orden realizada.
<i>Pedido</i>	idUsuario	Relaciona el pedido con el usuario que compró.
<i>Pedido_Producto</i>	idPedidoProducto	Identifica cada producto dentro del pedido.
<i>Pedido_Producto</i>	idPedido	Relaciona el producto con una orden.
<i>Pedido_Producto</i>	idProducto	Indica qué producto fue comprado.
<i>Pedido_Producto</i>	cantidad	Indica cuántas piezas se compraron.
<i>Pago</i>	idPago	Identifica cada registro de pago.
<i>Pago</i>	idPedido	Relaciona el pago con el pedido correspondiente.

6. Frameworks y tecnologías

Elemento	Tecnología	Uso en Pugnela
<i>Servidor web</i>	Apache HTTP Server	Permite alojar el sistema y procesar las solicitudes de usuarios y administradores.
<i>Lenguaje backend</i>	PHP	Se usa para programar la lógica del carrito, usuarios, pedidos, pagos y panel administrativo.
<i>Base de datos</i>	MySQL	Guarda usuarios, productos, carritos, pedidos y pagos.
<i>Arquitectura</i>	MVC	Separa modelos, vistas y controladores para organizar mejor el sistema.
<i>Control de versiones</i>	GitHub	Permite guardar avances, colaborar y controlar cambios del código.
<i>Gestión del proyecto</i>	Microsoft Project	Sirve para planear actividades, tiempos, responsables y avances.
<i>Apoyo de desarrollo</i>	Claude / IA	Ayuda con código, depuración, documentación y estructura del sistema.

7. Sprint del proyecto

Sprint 1: Base del sistema y estructura

Objetivo: crear la estructura inicial del sistema Pugnela.

Actividades:

- Crear carpetas del proyecto.
- Configurar Apache, PHP y MySQL.
- Crear la base de datos.
- Crear tablas principales.
- Crear conexión database.php.
- Crear estructura MVC.
- Subir avances a GitHub.

Duración sugerida: 1 semana.

Sprint 2: Módulo de usuario

Objetivo: permitir que el cliente navegue, vea productos y use el carrito.

Actividades:

- Crear página principal.
- Mostrar catálogo de roles.
- Crear vista de producto.
- Agregar productos al carrito.
- Modificar cantidades.
- Eliminar productos del carrito.
- Registrar usuarios.

Duración sugerida: 1 semana.

Sprint 3: Pedidos y pagos

Objetivo: permitir que el usuario finalice su compra.

Actividades:

- Convertir carrito en pedido.
- Guardar detalle del pedido.

- Calcular total.
- Registrar método de pago.
- Generar estado del pedido.
- Mostrar confirmación de compra.

Duración sugerida: 1 semana.

Sprint 4: Módulo administrativo

Objetivo: permitir que el administrador gestione productos y pedidos.

Actividades:

- Crear login de administrador.
- Agregar productos.
- Editar productos.
- Desactivar productos.
- Ver pedidos recibidos.
- Cambiar estado del pedido.
- Revisar pagos.

Duración sugerida: 1 semana.

8. Product Backlog

Prioridad	Funcionalidad	Descripción
<i>Alta</i>	Registro de usuario	Permitir que los clientes creen una cuenta.
<i>Alta</i>	Inicio de sesión	Permitir acceso seguro al sistema.
<i>Alta</i>	Catálogo de productos	Mostrar roles disponibles.
<i>Alta</i>	Carrito de compras	Permitir agregar, modificar y eliminar productos.
<i>Alta</i>	Generar pedido	Convertir el carrito en una orden.
<i>Alta</i>	Registro de pago	Guardar método, monto y estatus del pago.

<i>Media</i>	Panel administrativo	Gestionar productos y pedidos.
<i>Media</i>	Subir imágenes	Permitir agregar imágenes de productos.
<i>Media</i>	Estado del pedido	Cambiar pedido a pendiente, pagado o entregado.
<i>Baja</i>	Historial de compras	Mostrar pedidos anteriores al usuario.
<i>Baja</i>	Reportes básicos	Mostrar ventas y pedidos registrados.

9. Historias de usuario

ID	Historia de usuario	Criterio de aceptación
<i>HU01</i>	Como cliente, quiero registrarme para poder realizar compras en Pugnela.	El sistema debe guardar mis datos y permitirme iniciar sesión.
<i>HU02</i>	Como cliente, quiero ver el catálogo para elegir los roles disponibles.	El sistema debe mostrar nombre, imagen, descripción y precio.
<i>HU03</i>	Como cliente, quiero agregar productos al carrito para preparar mi compra.	El producto debe aparecer en el carrito con cantidad y precio.
<i>HU04</i>	Como cliente, quiero modificar cantidades para ajustar mi pedido.	El total debe actualizarse automáticamente.
<i>HU05</i>	Como cliente, quiero eliminar productos del carrito si cambio de opinión.	El producto debe desaparecer del carrito.
<i>HU06</i>	Como cliente, quiero finalizar mi compra para generar un pedido.	El sistema debe crear una orden con fecha, total y estado.
<i>HU07</i>	Como cliente, quiero registrar mi pago para confirmar mi compra.	El sistema debe guardar método de pago, monto y estatus.
<i>HU08</i>	Como administrador, quiero agregar productos para actualizar el catálogo.	El producto debe aparecer disponible para los clientes.
<i>HU09</i>	Como administrador, quiero editar productos para corregir información o precios.	Los cambios deben reflejarse en el catálogo.

<i>HU10</i>	Como administrador, quiero revisar pedidos para dar seguimiento a las ventas.	El sistema debe mostrar cliente, fecha, total y estado.
<i>HU11</i>	Como administrador, quiero cambiar el estado del pedido para controlar el proceso.	El pedido debe actualizarse como pendiente, pagado, enviado o entregado.
<i>HU12</i>	Como administrador, quiero desactivar productos para ocultarlos sin eliminarlos.	El producto ya no debe mostrarse al cliente.

10. Diagrama físico resumido

flowchart TD

A[Usuario] --> B[Carrito]

B --> C[Carrito_Producto]

C --> D[Producto]

A --> E[Pedido]

E --> F[Pedido_Producto]

F --> D

E --> G[Pago]

DIAGRAMA ENTIDAD-RELACIÓN (ER)

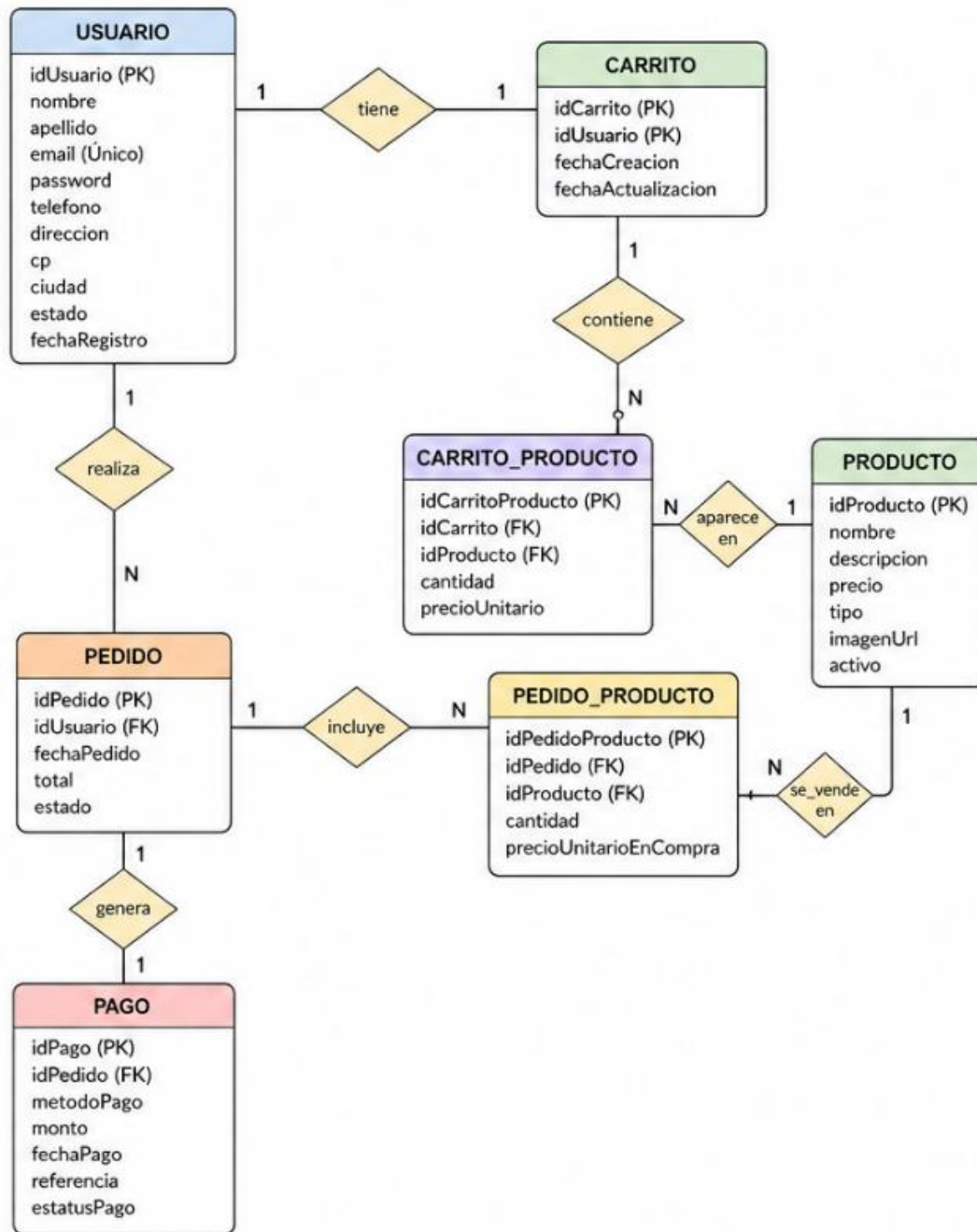
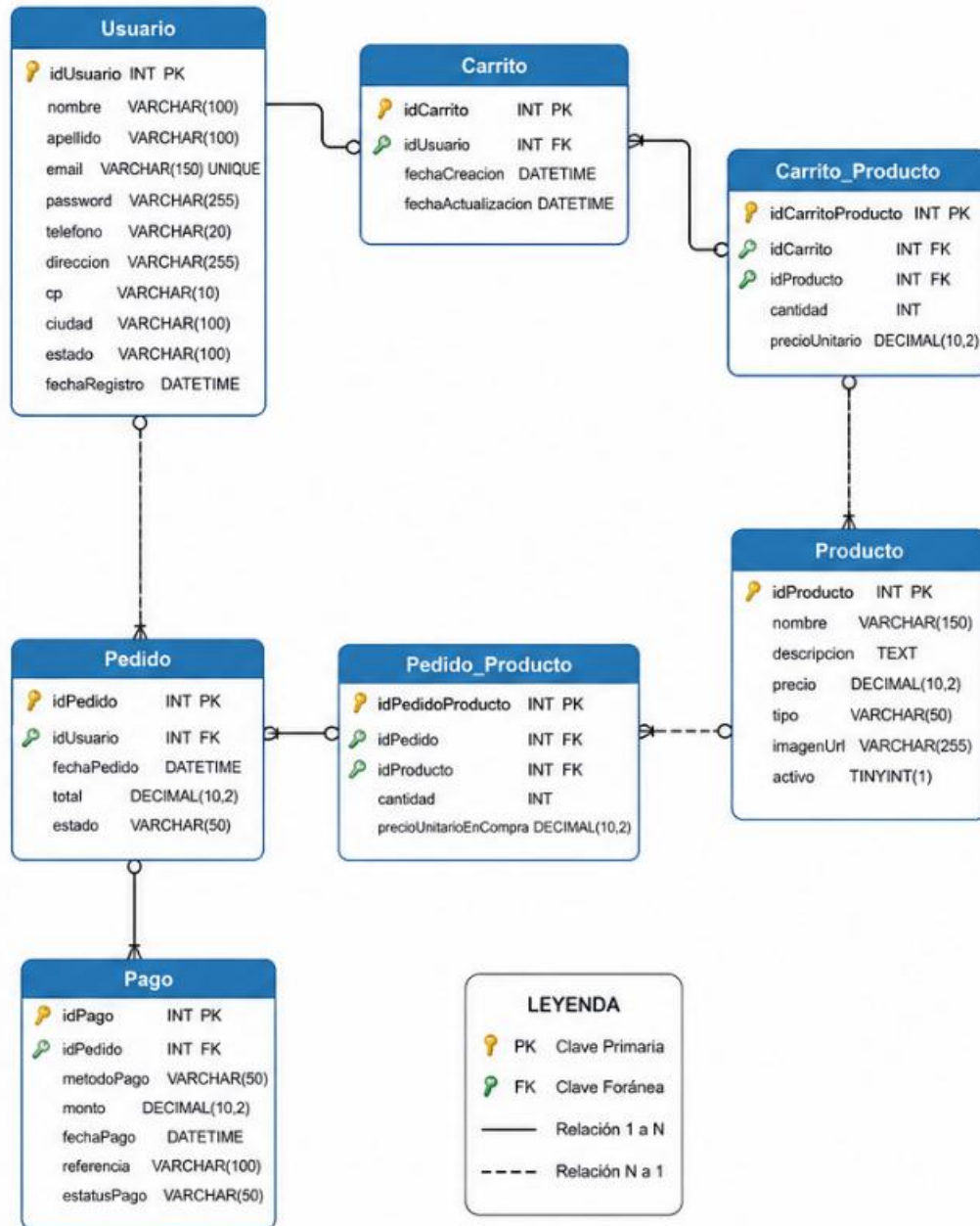


DIAGRAMA FÍSICO (BASE DE DATOS)



Mapa de navegación

